

A PROJECT REPORT ON

FACE RECOGNITION ATTENDANCE SYSTEM WITH ANTI-SPOOFING

Domain: Computer Vision / Education

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	13
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	22
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Attendance is an important part of academic discipline in schools and colleges, and minimum attendance is often a condition for appearing in examinations. The traditional method of calling out names takes five to ten minutes of every class period, which adds up to many hours of lost teaching time in a semester. Paper registers must also be manually entered into a computer to calculate percentages.

Proxy attendance, where a student answers on behalf of an absent friend, is common in large classes. Biometric fingerprint systems reduce proxies but create queues and require physical contact. Earlier face recognition attendance systems solved the contact problem but could be easily fooled by showing a printed photograph or a video of the student on a mobile phone in front of the camera.

This project, FaceRoll, marks attendance automatically using a classroom camera. Faces are detected in the live video, a liveness check rejects printed photos and phone screens, deep face embeddings are generated using FaceNet/ArcFace models, and each face is matched with the enrolled students using cosine similarity. Attendance is recorded with a timestamp and spoof attempts are logged.

LIMITATIONS

1. Manual roll call wastes valuable class time.
2. Proxy attendance is widespread in large classes.
3. Existing face systems are easily fooled by photos and videos.
4. Fingerprint systems require contact and create queues.

1.1 History of Domain

Automatic face recognition research began in the 1960s with Woody Bledsoe's manual landmark-based system. In 1991 Turk and Pentland introduced Eigenfaces based on Principal Component Analysis, followed by Fisherfaces and Local Binary Pattern Histograms. The Viola-Jones detector (2001) made real-time face detection possible.

Deep learning transformed the field: DeepFace (Facebook, 2014) and FaceNet (Google, 2015) achieved near-human accuracy on the Labeled Faces in the Wild benchmark by learning compact face embeddings. ArcFace (2019) further improved discrimination using an additive angular margin loss. At the same time, presentation attack detection (anti-spoofing) emerged as a research area using texture analysis, eye-blink detection and depth cues.

1.2 Overview of the Project

During enrolment, each student's face is captured from several angles and the embeddings are stored. During a class, the camera streams video to the FaceRoll server. Faces are detected with MTCNN/RetinaFace, and each face goes through a liveness check that combines eye-blink detection (Eye Aspect Ratio) over a few seconds with a texture-based classifier trained to distinguish real skin from printed paper and screens.

Live faces are converted into 512-dimensional embeddings with ArcFace and compared with stored embeddings. If the highest cosine similarity exceeds the threshold, the student is marked present for that period. Unknown faces and spoof attempts are saved with a snapshot for the faculty to review. Attendance reports can be exported in Excel or PDF format.

1.3 Motivation

Teachers want to spend class time teaching, not taking attendance, and institutions need honest attendance data.

- **Motivation:** To automate attendance in a contactless way that cannot be fooled by photographs or videos.
- **Outcome:** Accurate, proxy-free attendance with zero class time wasted and instant reports.

1.4 Aim of the Project

The project is entitled as “**FACE RECOGNITION ATTENDANCE SYSTEM WITH ANTI-SPOOFING**”. The aim of the project is to develop an automatic attendance system that recognises students from a classroom camera using deep face embeddings and prevents proxy attendance using liveness detection.

1.5 Objectives

- To enrol students with multiple face images.
- To detect faces in live video in real time.
- To implement liveness detection to reject photo and screen attacks.
- To recognise faces using embeddings and cosine similarity.
- To store attendance and generate reports.

1.6 Literature Review

Turk and Pentland (1991): Proposed the Eigenfaces approach in which face images are projected onto a low-dimensional space found by PCA and recognition is performed by distance in this space.

Viola and Jones (2001): Introduced a rapid object detection framework using Haar-like features, integral images and a cascade of AdaBoost classifiers, enabling real-time face detection.

Schroff, Kalenichenko and Philbin (2015): Presented FaceNet, which learns a mapping from face images to a compact Euclidean space using triplet loss, achieving 99.63% accuracy on LFW.

Zhang et al. (2016): Proposed MTCNN, a multi-task cascaded convolutional network that jointly performs face detection and facial landmark alignment.

Deng et al. (2019): Proposed ArcFace, an additive angular margin loss that produces highly discriminative face embeddings and achieved state-of-the-art results on several benchmarks.

Soukupova and Cech (2016): Proposed real-time eye blink detection using facial landmarks and the Eye Aspect Ratio (EAR), which is used here as a liveness cue.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, attendance is taken by calling names or passing a sheet for signatures. Some institutions use fingerprint devices at the entrance. A few use basic face recognition based on Haar cascades and LBPH, which have low accuracy in varying light and do not detect spoofing.

DISADVANTAGES

- Time-consuming manual process.
- Proxy attendance is easy.
- Fingerprint devices create queues and hygiene concerns.
- Basic face systems accept photographs.
- Manual data entry for reports.

2.2 Proposed System

The proposed FaceRoll system uses deep learning for face detection (MTCNN/RetinaFace) and recognition (ArcFace embeddings), combined with a two-stage liveness check (eye-blink and texture classifier). Attendance is marked automatically with timestamp, spoof attempts are logged with images, and reports are generated automatically.

ADVANTAGES

- Contactless and automatic.
- Prevents proxies and photo/video attacks.
- High accuracy under different lighting.
- Saves class time.
- Instant reports and low-attendance alerts.

2.3 Project Schedule

The development of FaceRoll follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the FaceRoll system are listed below.

- **Student enrolment:** Capture 10-20 images per student and store embeddings.
- **Face detection:** Detect all faces in each video frame.
- **Liveness detection:** Reject photos and screens using blink and texture analysis.
- **Recognition:** Match embeddings with enrolled students.
- **Attendance marking:** Mark present with date, period and time.
- **Spoof logging:** Save snapshots of unknown and spoof faces.
- **Reports:** Export daily and monthly attendance in Excel/PDF.
- **Alerts:** Notify students below 75% attendance.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Accuracy:** Recognition accuracy above 97% for enrolled students.
- **Performance:** Process at least 10 frames per second.
- **Security:** Face embeddings stored, not raw images, with access control.

- **Privacy:** Data used only for attendance; consent taken during enrolment.
- **Usability:** One-click start of a class session.
- **Reliability:** Works under normal classroom lighting.

3.3 User Requirements

- Faculty can start attendance for a period with one click
- Students are recognised without any action
- Faculty can correct attendance manually
- Reports available instantly
- Spoof attempts visible to faculty

3.4 System Requirements

- HD webcam or IP camera in the classroom
- Server with Python, OpenCV and DeepFace
- MySQL database
- Flask web application
- Local network connectivity

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above
Hard Disk : 256 GB SSD
RAM : 8 GB
GPU : Optional NVIDIA GPU for faster recognition
Camera : 1080p USB webcam / IP camera
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Windows 10 / Ubuntu 22.04
Coding Language : Python 3.10
Computer Vision : OpenCV, dlib, DeepFace (ArcFace)
Face Detection : MTCNN / RetinaFace
Back End : Flask
Front End : HTML, Bootstrap, JavaScript
Database : MySQL

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed FaceRoll system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Pre-trained face detection and recognition models are freely available through DeepFace and dlib, and work on a normal laptop CPU at classroom scale. OpenCV provides camera capture. Therefore, the project is technically feasible.

ECONOMIC FEASIBILITY

The system needs only a webcam (about Rs. 2,000) per classroom and a computer that most colleges already have. All software is open source. It is economically feasible.

OPERATIONAL FEASIBILITY

Faculty only click 'Start Attendance' and 'Stop'. Students do nothing special. Reports are automatic. The system is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI),

which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

OPENCV

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library containing more than 2500 optimised algorithms. It was originally developed by Intel and is now maintained by the OpenCV foundation.

OpenCV is used to capture video from cameras, resize and convert images, detect edges and contours, apply filters and run deep neural network models through its dnn module.

- Real-time image and video processing
- Python, C++ and Java interfaces
- Hardware acceleration support

DLIB / DEEPPFACE

Dlib is a modern C++ toolkit containing machine learning algorithms, including a robust face detector and 68-point facial landmark predictor. DeepFace is a lightweight Python face recognition framework that wraps state-of-the-art models such as FaceNet, ArcFace and VGG-Face.

These libraries convert a face image into a fixed-length numeric embedding vector. Two images of the same person produce embeddings that are close together, so recognition is performed by comparing distances between vectors.

FLASK

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

MYSQL

MySQL is an open-source relational database management system (RDBMS) based on Structured Query Language. It is one of the most widely used databases in web applications and is known for its speed, reliability and ease of use.

MySQL stores data in tables made of rows and columns, enforces relationships using primary and foreign keys, and supports transactions through the InnoDB storage engine.

- Fast and reliable
- Easy to install and manage with tools like MySQL Workbench
- Supported by all major programming languages

BOOTSTRAP

Bootstrap is a free and open-source CSS framework directed at responsive, mobile-first front-end web development. It contains HTML, CSS and JavaScript-based design templates for typography, forms, buttons, navigation and other interface components.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of FaceRoll is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

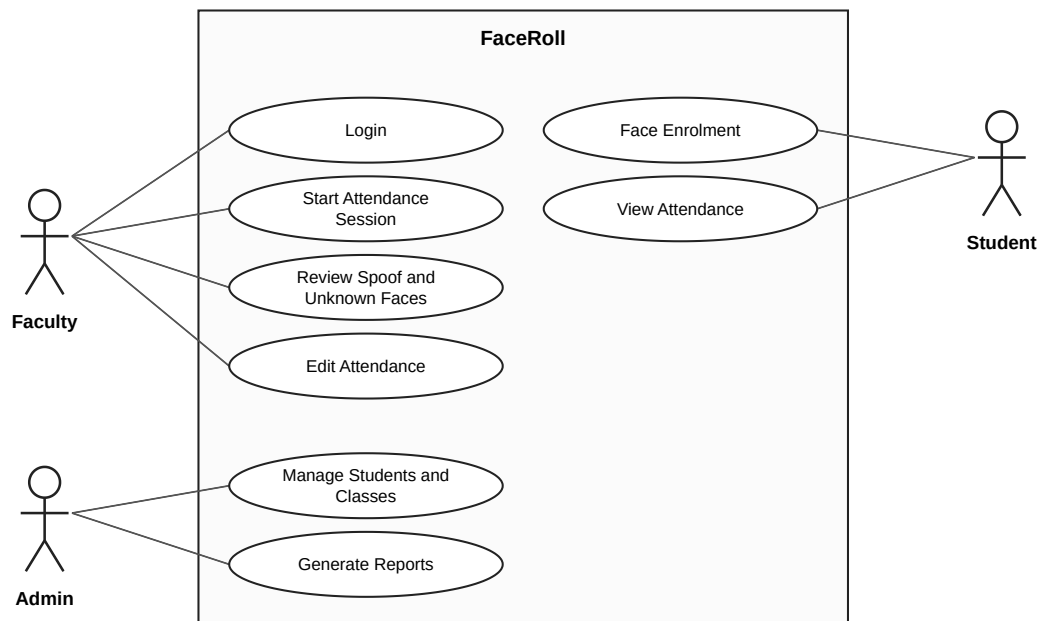


Fig 1: Use case diagram of FaceRoll

6.2 System Architecture

FaceRoll consists of a capture layer (camera), a vision processing pipeline (detection, liveness, recognition), the Flask application and a MySQL database.

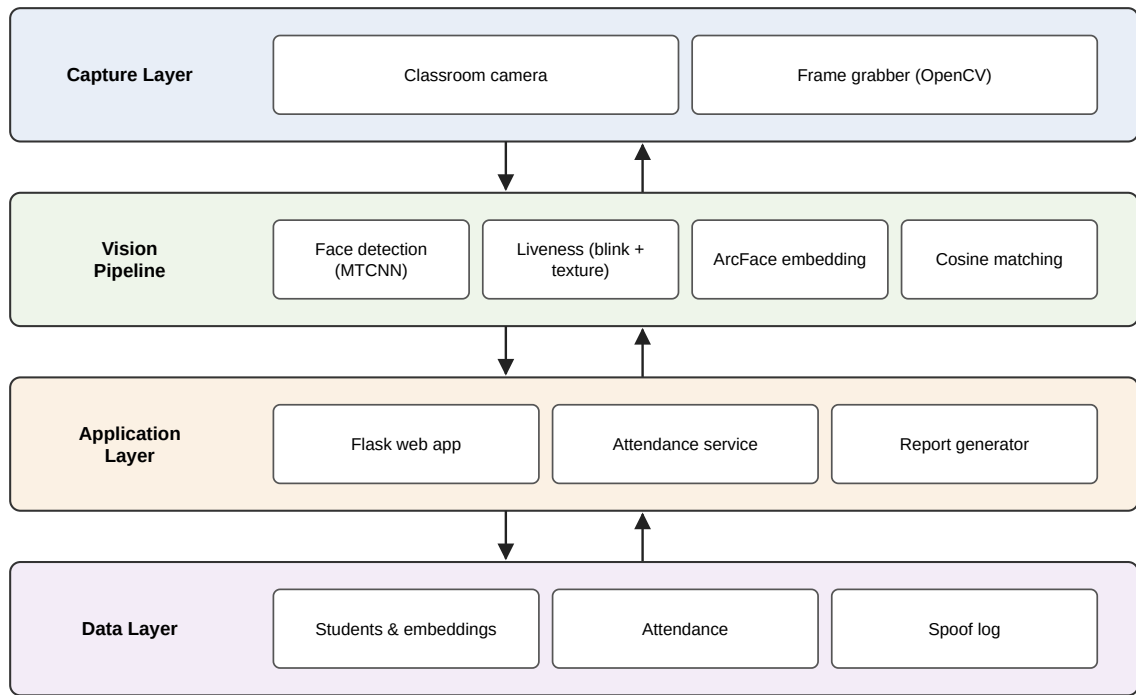


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

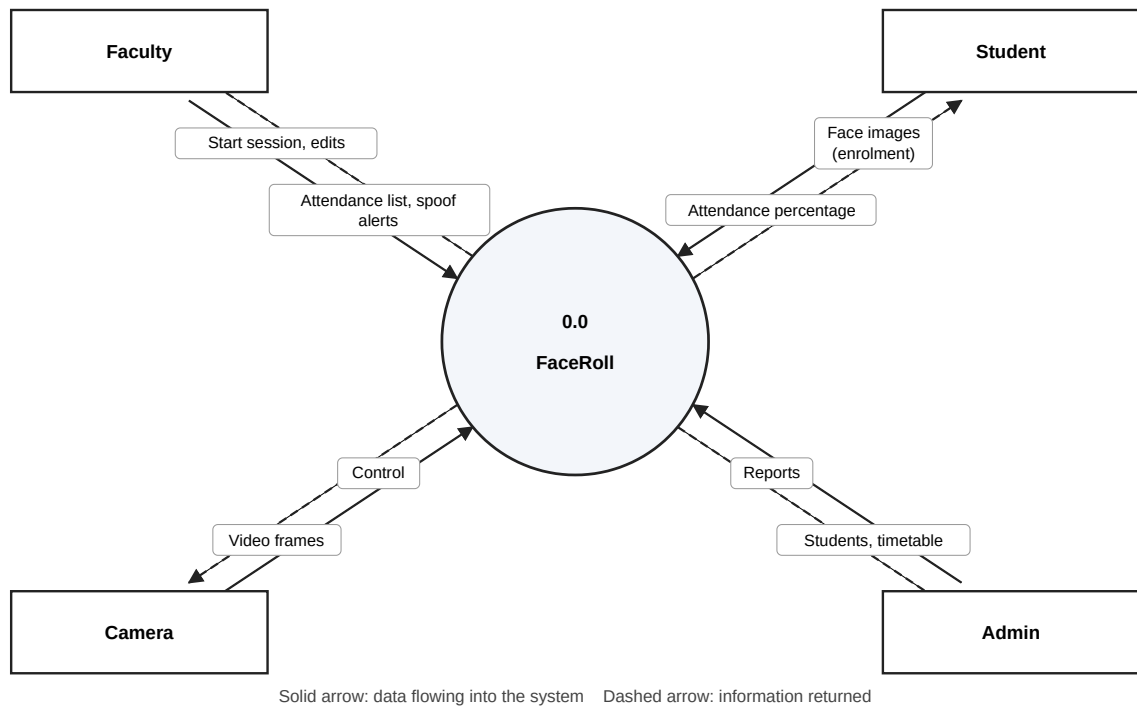


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

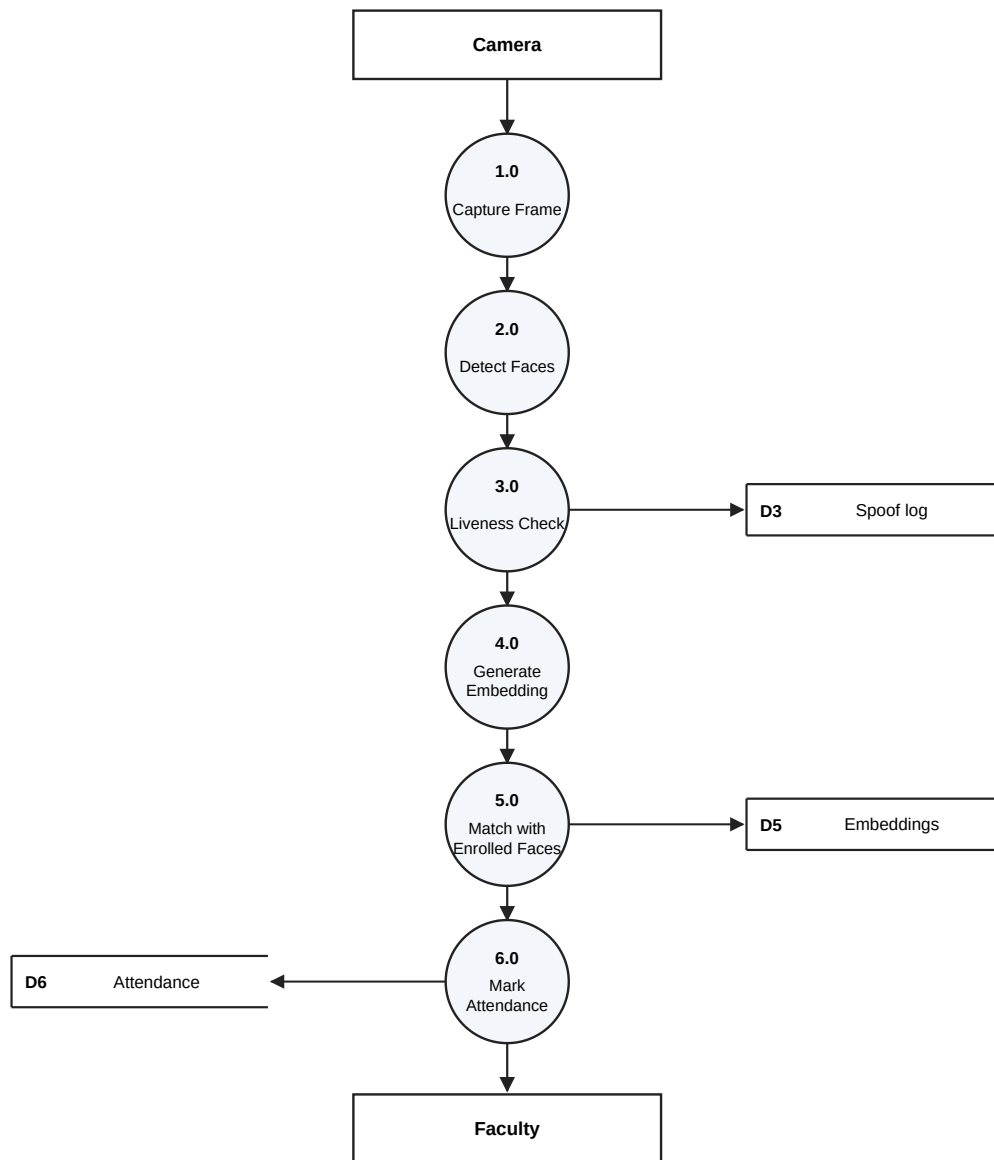


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

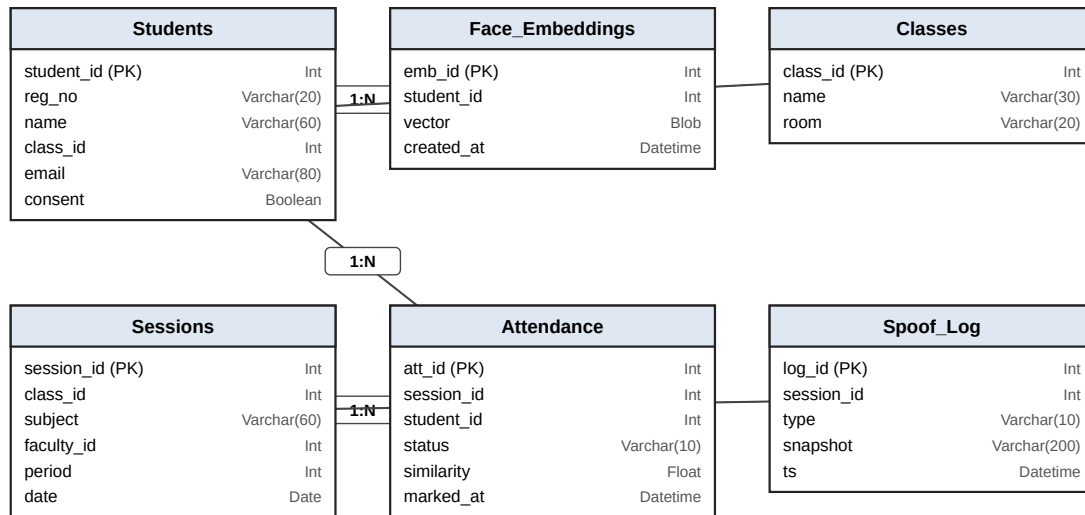


Fig 5: Entity relationship diagram

6.5 Table Design

STUDENTS

FIELD NAME	DATA TYPE	DESCRIPTION
student_id	Int	Primary key
reg_no	Varchar(20)	Register number
name	Varchar(60)	Student name
class_id	Int	Foreign key to Classes
email	Varchar(80)	E-mail
consent	Boolean	Enrolment consent

FACE_EMBEDDINGS

FIELD NAME	DATA TYPE	DESCRIPTION
emb_id	Int	Primary key
student_id	Int	Foreign key to Students
vector	Blob	512-d ArcFace embedding
created_at	Datetime	Enrolment time

CLASSES

FIELD NAME	DATA TYPE	DESCRIPTION
------------	-----------	-------------

class_id	Int	Primary key
name	Varchar(30)	e.g. III B.Sc CS A
room	Varchar(20)	Room number

SESSIONS

FIELD NAME	DATA TYPE	DESCRIPTION
session_id	Int	Primary key
class_id	Int	Foreign key to Classes
subject	Varchar(60)	Subject
faculty_id	Int	Faculty
period	Int	Period number
date	Date	Date

ATTENDANCE

FIELD NAME	DATA TYPE	DESCRIPTION
att_id	Int	Primary key
session_id	Int	Foreign key to Sessions
student_id	Int	Foreign key to Students
status	Varchar(10)	present / absent / OD
similarity	Float	Match score
marked_at	Datetime	Time

SPOOF_LOG

FIELD NAME	DATA TYPE	DESCRIPTION
log_id	Int	Primary key
session_id	Int	Foreign key to Sessions
type	Varchar(10)	spoof / unknown
snapshot	Varchar(200)	Image path
ts	Datetime	Time

6.6 Algorithm Design

The attendance pipeline processes every frame captured during the first minutes of a period.

1. **Enroll students with multiple face images** – 15 images per student with different angles and expressions; the mean embedding is also stored.
2. **Detect faces in live video (MTCNN/RetinaFace)** – Faces smaller than 60 pixels are ignored.
3. **Run liveness check (blink/texture analysis)** – EAR below 0.21 for 2-3 frames counts as a blink; an LBP + SVM texture classifier gives a real-face probability.
4. **Generate face embeddings (FaceNet/ArcFace)** – Aligned 112x112 face crops are converted into 512-d normalised vectors.
5. **Match against database using cosine similarity** – The student with the highest similarity above 0.45 (ArcFace) is selected.
6. **Mark attendance with timestamp** – A student is marked present once per session.
7. **Reject unknown or spoof faces and log them** – Snapshots are stored for review.
8. **Export attendance reports** – Pandas generates Excel and PDF reports.

PSEUDO CODE

```
FOR each frame in session (first 10 minutes)
  faces <- DETECT(frame)
  FOR face in faces
    track <- TRACKER.update(face)
    IF NOT track.blinked AND EAR(face) < 0.21 THEN track.blinked <- TRUE
    live <- track.blinked AND TEXTURE_SVM(face) > 0.5
    IF NOT live AND track.age > 90 frames THEN LOG_SPOOF(face); CONTINUE
    e <- ARCFACE(ALIGN(face))
    (sid, s) <- MAX over enrolled: COSINE(e, emb[sid])
    IF s > 0.45 AND live THEN MARK_PRESENT(sid, session, now)
    ELSE IF s <= 0.45 THEN LOG_UNKNOWN(face)
```

6.7 Modules

The modules of FaceRoll are grouped by the Faculty, Student and Admin users.

MODULES

FACULTY

1. LOGIN
2. START ATTENDANCE SESSION
3. REVIEW SPOOF AND UNKNOWN FACES
4. EDIT ATTENDANCE

STUDENT

1. FACE ENROLMENT
2. VIEW ATTENDANCE

ADMIN

1. MANAGE STUDENTS AND CLASSES
2. GENERATE REPORTS

FACULTY

LOGIN

Faculty log in with staff ID and password to access their timetable and classes.

START ATTENDANCE SESSION

Faculty select the class and period and start the camera. The system marks attendance continuously for the first 10 minutes of the period.

REVIEW SPOOF AND UNKNOWN FACES

Faculty view snapshots of faces flagged as spoof or unknown and can take disciplinary action.

EDIT ATTENDANCE

Faculty can manually mark a student present or absent with a reason (for example, on duty).

STUDENT

FACE ENROLMENT

With consent, the student's face is captured from different angles; embeddings are stored against the register number.

VIEW ATTENDANCE

Students view their subject-wise attendance percentage.

ADMIN

MANAGE STUDENTS AND CLASSES

Admin adds students, classes, subjects and the timetable.

GENERATE REPORTS

Admin generates department-wise attendance reports and shortage lists.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

FaceRoll was implemented in Python using OpenCV for video capture, dlib for 68-point landmarks used in blink detection, and DeepFace for ArcFace embeddings and RetinaFace detection. The Flask web application streams the annotated video to the browser using MJPEG, and attendance is stored in MySQL.

7.2 Implementation

The texture-based anti-spoofing classifier was trained on the Replay-Attack style dataset plus 600 locally captured samples of printed photos and phone screens, using uniform LBP histograms and an RBF SVM. The recognition threshold was tuned on a validation set of 60 students to balance false accepts and false rejects.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Enrol student	15 images of student	Embeddings stored	Pass
TC02	Recognise enrolled student	Student in front of camera	Marked present with name	Pass
TC03	Printed photo attack	A4 photo of student	Spoof detected, not marked	Pass
TC04	Phone screen attack	Video on mobile	Spoof detected and logged	Pass
TC05	Unknown person	Visitor face	Logged as unknown	Pass
TC06	Multiple faces	20 students in frame	All recognised in < 5 s	Pass
TC07	Duplicate marking	Same student in many frames	Marked only once	Pass
TC08	Report export	Month = October	Excel file with percentages	Pass

PERFORMANCE EVALUATION

Evaluation on 60 enrolled students over 20 sessions:

Metric	Value
Recognition accuracy	98.4%
False acceptance rate	0.3%
Spoof detection rate (photo)	97.0%
Spoof detection rate (screen)	94.5%
Average time to mark a class of 60	38 seconds

7.4 Source Code

enroll.py

```
import cv2, os, numpy as np, mysql.connector
from deepface import DeepFace

db = mysql.connector.connect(user="facereoll", password="secret",
                             database="facereoll")

def enroll(student_id, n=15):
    cam = cv2.VideoCapture(0); vectors = []
    while len(vectors) < n:
        ok, frame = cam.read()
        if not ok: break
        try:
            rep = DeepFace.represent(frame, model_name="ArcFace",
                                     detector_backend="retinaface")
        except ValueError:
            continue # no face found
        if len(rep) == 1:
            v = np.array(rep[0]["embedding"], dtype=np.float32)
            vectors.append(v / np.linalg.norm(v))
            cv2.putText(frame, f"Captured {len(vectors)}/{n}", (20, 40), 0, 1,
                        (0, 255, 0), 2)
            cv2.imshow("Enrolment - turn your head slowly", frame); cv2.waitKey(300)
            cam.release(); cv2.destroyAllWindows()
            cur = db.cursor()
            for v in vectors + [np.mean(vectors, axis=0)]:
                cur.execute("INSERT INTO face_embeddings(student_id, vector, created_at)
VALUES (%s,%s,NOW())",
                            (student_id, v.tobytes()))
            db.commit()
```

attendance.py

```
import cv2, dlib, joblib, numpy as np, mysql.connector, time
from scipy.spatial import distance
from skimage.feature import local_binary_pattern
from deepface import DeepFace

predictor = dlib.shape_predictor("models/shape_predictor_68_face_landmarks.dat")
texture_svm = joblib.load("models/lbp_svm.pkl")
db = mysql.connector.connect(user="facereoll", password="secret",
                             database="facereoll")
THRESH, EAR_T = 0.45, 0.21

def load_embeddings(class_id):
    cur = db.cursor()
    cur.execute("""SELECT e.student_id, e.vector FROM face_embeddings e JOIN
students s
                ON s.student_id=e.student_id WHERE s.class_id=%s""",
(class_id,))
    ids, vecs = zip(*[(sid, np.frombuffer(v, np.float32)) for sid, v in
cur.fetchall()])
    return np.array(ids), np.vstack(vecs)

def ear(eye):
    return (distance.euclidean(eye[1], eye[5]) + distance.euclidean(eye[2],
```

```

eye[4])) / (2 * distance.euclidean(eye[0], eye[3]))

def texture_live(face):
    g = cv2.resize(cv2.cvtColor(face, cv2.COLOR_BGR2GRAY), (64, 64))
    hist, _ = np.histogram(local_binary_pattern(g, 8, 1, "uniform"), bins=10,
range=(0, 10), density=True)
    return texture_svm.predict_proba([hist])[0, 1] > 0.5

def run_session(session_id, class_id, minutes=10):
    ids, embs = load_embeddings(class_id)
    marked, blinked = set(), set()
    cam = cv2.VideoCapture(0); end = time.time() + minutes * 60
    while time.time() < end:
        ok, frame = cam.read()
        if not ok: continue
        faces = DeepFace.extract_faces(frame, detector_backend="retinaface",
enforce_detection=False)
        for f in faces:
            a = f["facial_area"]; x, y, w, h = a["x"], a["y"], a["w"], a["h"]
            if w < 60: continue
            crop = frame[y:y+h, x:x+w]
            pts = predictor(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY),
dlib.rectangle(x, y, x+w, y+h))
            p = np.array([[pts.part(i).x, pts.part(i).y] for i in range(68)])
            key = (x // 40, y // 40) # simple position-
based track key
            if (ear(p[36:42]) + ear(p[42:48])) / 2 < EAR_T:
                blinked.add(key)
            if not (key in blinked and texture_live(crop)):
                continue
            v = np.array(DeepFace.represent(crop, model_name="ArcFace",
detector_backend="skip")[0]["embedding"])
            sims = embs @ (v / np.linalg.norm(v))
            best = int(np.argmax(sims))
            if sims[best] > THRESH and ids[best] not in marked:
                marked.add(ids[best])
                db.cursor().execute("INSERT INTO attendance(session_id,
student_id, status, similarity, marked_at) "
                                "VALUES (%s,%s,'present',%s,NOW())",
(session_id, int(ids[best]), float(sims[best])))
                db.commit()
        cam.release()
    return marked

```

7.5 Screen Layout

SCREENSHOTS:

LIVE CLASS ATTENDANCE

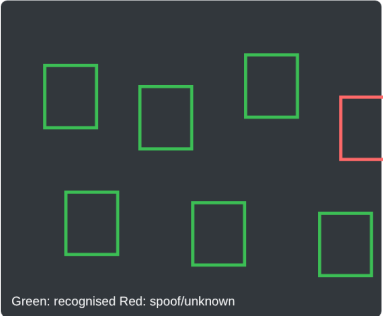
https://faceroll.app/

FaceRoll

Live ClassEnrollAttendanceSpoof LogReports

III B.Sc CS - A | Data Mining | Period 2

Camera



Green: recognised Red: spoof/unknown

Session running - 52 / 60 present

Started	10:05 AM
Recognised	52
Spoof attempts	1 (phone screen)
Unknown faces	0
Auto-stop	10:15 AM

Latest: 21UCS1045 Priya R - 10:06:12 (0.71)

Latest: 21UCS1012 Arun K - 10:06:15 (0.68)

STUDENT ENROLMENT

https://faceroll.app/

FaceRoll

Live ClassEnrollAttendanceSpoof LogReports

Enrol student face

Register Number

21UCS1045

Student Name

Priya Ramesh

Class

III B.Sc CS - A

Consent

Yes - signed consent form received

Start Capture (15 images)

Instructions

Sit about 1 metre from the camera.

Turn your head slowly left, right, up and down.

Remove mask; spectacles are allowed.

Ensure the face is well lit.

ATTENDANCE REGISTER

https://faceroll.app/

FaceRoll

Live Class Enroll Attendance Spoof Log Reports

Attendance - 14 Oct 2026 - Period 2

Class: III B.Sc CS - A Subject: Data Mining

Reg. No	Name	Time	Score	Status
21UCS1001	Abinaya S	10:05:40	0.74	PRESENT
21UCS1012	Arun K	10:06:15	0.68	PRESENT
21UCS1023	Deepak M	-	-	ABSENT
21UCS1045	Priya R	10:06:12	0.71	PRESENT
21UCS1050	Sanjay V	-	-	OD

SPOOF LOG

https://faceroll.app/

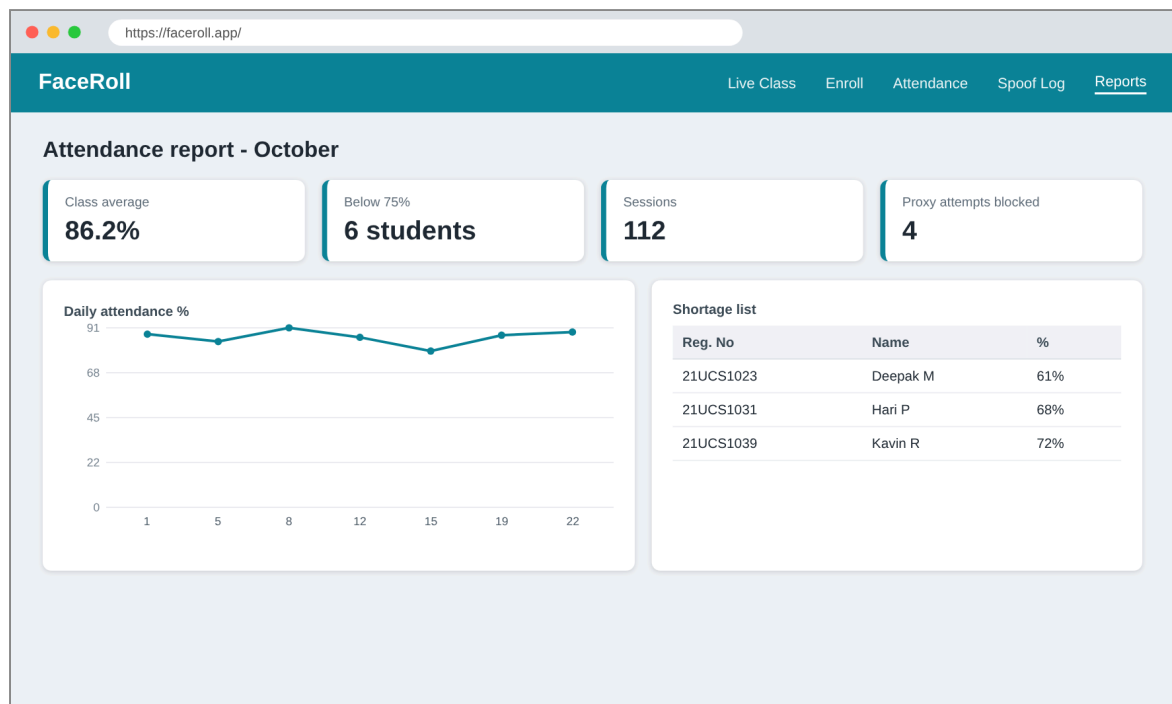
FaceRoll

Live Class Enroll Attendance Spoof Log Reports

Spoof and unknown face log

Time	Class	Type	Reason	Action
14-10 10:07	III CS-A	SPOOF	Screen texture, no blink	Reviewed
13-10 14:02	II CS-B	SPOOF	Printed photo	Warning issued
12-10 09:10	III CS-A	UNKNOWN	Not enrolled	Guest lecturer

MONTHLY REPORT



CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Face Recognition Attendance System with Anti-Spoofing automates classroom attendance with a recognition accuracy of 98.4% and blocks most photo and screen attacks using a combination of blink detection and texture analysis. A class of 60 students is marked within about 40 seconds without any action from students or faculty.

The system saves teaching time, eliminates proxy attendance and produces instant reports, making it a practical solution for colleges and offices.

8.2 Future Enhancement

- Depth-based anti-spoofing using stereo or infrared cameras.
- Mask-aware recognition models.
- Edge deployment on Jetson Nano or Raspberry Pi 5 per classroom.
- Integration with the college ERP and parent SMS alerts.
- Emotion and engagement analytics (with consent).

CHAPTER IX

BIBLIOGRAPHY

1. Turk, M., and Pentland, A., "Eigenfaces for Recognition", Journal of Cognitive Neuroscience, Vol. 3, No. 1, pp. 71-86, 1991.
2. Viola, P., and Jones, M., "Rapid Object Detection using a Boosted Cascade of Simple Features", Proc. IEEE CVPR, 2001.
3. Schroff, F., Kalenichenko, D., and Philbin, J., "FaceNet: A Unified Embedding for Face Recognition and Clustering", Proc. IEEE CVPR, 2015.
4. Zhang, K., Zhang, Z., Li, Z., and Qiao, Y., "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks", IEEE Signal Processing Letters, Vol. 23, No. 10, 2016.
5. Deng, J., Guo, J., Xue, N., and Zafeiriou, S., "ArcFace: Additive Angular Margin Loss for Deep Face Recognition", Proc. IEEE CVPR, 2019.
6. Soukupova, T., and Cech, J., "Real-Time Eye Blink Detection using Facial Landmarks", 21st Computer Vision Winter Workshop, 2016.

WEBSITE REFERENCE

1. <https://opencv.org>
2. <https://github.com/serengil/deepface>
3. <http://dlib.net>
4. <https://flask.palletsprojects.com>